# Next-Generation Logic Programming: Beyond Prolog in a Propagator-First Architecture

Prologos Research

2026-03-16

## Contents

**1 Introduction** **4**
1.1 Existing Infrastructure Reference . . . . . 5

**2 Mode Prefixes on Logic Variables** **5**
2.1 Background: What Modes Are . . . . . 5
2.2 Mercury's Mode System . . . . . 5
2.3 Ciao's Assertion Language . . . . . 6
2.4 Proposed Prologos Syntax . . . . . 6
2.5 Determinism Inference from Modes . . . . . 7
2.6 Composition with the Propagator Network . . . . . 7
2.7 Implementation Roadmap . . . . . 7

**3 Domain-Constrained Logic Variables** **8**
3.1 Background: CLP(X) Family . . . . . 8
3.2 Proposed Prologos Syntax . . . . . 8
3.3 Arc Consistency and Bounds Propagation as Propagator Cells 8
3.4 Lattice Instances for Constraint Domains . . . . . 9
3.5 Composition with ATMS . . . . . 9
3.6 Implementation Roadmap . . . . . 9

**4 Co-Inductive Logic Programming and Greatest Fixpoints** **10**
4.1 Background: Coinduction as Dual of Induction . . . . . 10
4.2 CoLP: Coinductive Hypothesis Rule . . . . . 10
4.3 Flexible Coinduction: Coclauses . . . . . 11
4.4 Composition with Bilattice (Well-Founded Engine) . . . . . 11
4.5 Coinductive Session Types . . . . . 12
4.6 Implementation Roadmap . . . . . 13

**5 Rule Learning and Inductive Logic Programming** **13**
5.1 Background: ILP Systems . . . 13
5.2 Meta-Interpretive Learning (MIL) . . . 13
5.3 Proposed Prologos Surface Syntax for ILP . . . 14
5.4 Differentiable ILP . . . 14
5.5 Composition with Propagator Networks . . . 15
5.6 Implementation Roadmap . . . 15

**6 Probabilistic Logic Programming** **16**
6.1 Background: ProbLog and the Distribution Semantics . . . 16
6.2 Algebraic ProbLog and Semirings . . . 16
6.3 DeepProbLog: Neural Predicates . . . 16
6.4 DC-ProbLog: Continuous Distributions (2024) . . . 17
6.5 Proposed Prologos Syntax . . . 17
6.6 Semiring Framework as Propagator Lattice . . . 18
6.7 Knowledge Compilation and the ATMS . . . 18
6.8 Implementation Roadmap . . . 19

**7 Category-Theoretic Foundations** **19**
7.1 Why Category Theory Matters for Prologos . . . 19
7.2 Toposes as Generalized Logic . . . 19
7.3 Sheaves for Contextual Reasoning . . . 20
7.4 Kan Extensions for Program Transformation . . . 21
7.5 Galois Connections for Abstract Interpretation . . . 21
7.6 Fibered Categories for Dependent Types + Logic . . . 22
7.7 Institutions for Multi-Logic Reasoning . . . 23
7.8 Implementation Roadmap . . . 23

**8 Novel Directions** **24**
8.1 Linear Logic Programming . . . 24
8.2 Temporal Logic Programming . . . 25
8.3 Epistemic Logic Programming . . . 25
8.4 Abductive Logic Programming . . . 26
8.5 Answer Set Programming (ASP) Integration . . . 27
8.6 Constraint Handling Rules (CHR) . . . 27
8.7 Advanced Tabling: Beyond SLG . . . 28
8.7.1 Subsumptive Tabling . . . 28
8.7.2 Mode-Directed Tabling . . . 28
8.7.3 Magic Sets . . . 29
8.8 Implementation Priority Matrix . . . 29

**9 The Frontier: Modern LP Redesign (2020–2026) 29**
9.1 Survey of Recent Systems . . . 29
9.1.1 Flix (Aarhus/Waterloo, 2016–2025) . . . 29
9.1.2 Formulog (Harvard, 2020–2024) . . . 30
9.1.3 Ascent (2022–2024) . . . 30
9.1.4 Crepe (2021–2024) . . . 30
9.1.5 Flan (Purdue, POPL 2024) . . . 31
9.1.6 Soufflé (2016–2025) . . . 31
9.2 What These Systems Teach Us . . . 31
9.3 The Propagator Network as Universal Substrate . . . 32

**10 Composition Map 32**
10.1 Pairwise Composition Matrix . . . 32
10.2 Key Composition Synergies . . . 33
10.2.1 Modes + Domains (Strong) . . . 33
10.2.2 Probabilistic + Domains (Strong) . . . 33
10.2.3 Probabilistic + ILP (Strong) . . . 33
10.2.4 Coinduction + Category Theory (Strong) . . . 33
10.2.5 Modes + Linear Types (Strong) . . . 33
10.2.6 Category Theory + Probabilistic (Strong) . . . 34
10.3 Composition with Existing Prologos Features . . . 34
10.3.1 With Dependent Types . . . 34
10.3.2 With Session Types . . . 34
10.3.3 With the Propagator Network . . . 34
10.3.4 With the ATMS . . . 34

**11 Research References 35**
11.1 Mode Analysis and Optimization . . . 35
11.2 Coinductive Logic Programming . . . 35
11.3 Probabilistic Logic Programming . . . 35
11.4 Inductive Logic Programming . . . 36
11.5 Category Theory and Logic . . . 36
11.6 Well-Founded Semantics and Bilattices . . . 37
11.7 Modern Datalog Systems . . . 37
11.8 Linear Logic and Session Types . . . 37
11.9 ASP and CHR . . . 38
11.10 Abductive Logic Programming . . . 38
11.11 Tabling . . . 38

**12 Appendix: Lattice Requirements per Extension 38**

**13 Appendix: Implementation Priority** **39**
13.1 Tier 1: Low-Hanging Fruit (High Reuse, Moderate Effort) . . 39
13.2 Tier 2: Medium Effort, High Impact . . . . . 39
13.3 Tier 3: High Effort, Transformative Impact . . . . . 40
13.4 Tier 4: Research-Grade . . . . . 40

# 1 Introduction

Prologos already has a relational sublanguage (`defr`, `rel`, `solve`, `explain`) backed by a persistent propagator network, bilattice-based well-founded semantics, SLG-style tabling, ATMS-based assumption management, and narrowing. This document explores how to extend this foundation beyond a Prolog port into a genuinely next-generation logic programming system.

The key insight is that Prologos's propagator-first architecture is not merely an implementation detail—it is a *design space multiplier*. Where classical LP systems must bolt on constraint solvers, probabilistic inference, or coinductive reasoning as special-case extensions, a propagator network with pluggable lattices can express all of these as instances of the same fixpoint computation.

This document surveys seven major extension axes and one integrative frontier:

1. Mode prefixes on logic variables (`+`, `-`, `?`)
2. Domain-constrained logic variables (`?x:DomainType`)
3. Co-inductive logic programming and greatest fixpoints
4. Rule learning and inductive logic programming
5. Probabilistic logic programming
6. Category-theoretic foundations
7. Novel directions (linear LP, temporal LP, epistemic LP, abduction, ASP, CHR, tabling)
8. The frontier: what a fresh redesign of LP looks like

Each section connects back to Prologos's existing infrastructure with concrete implementation considerations.

### 1.1 Existing Infrastructure Reference

| Module | Role | Key Abstractions |
|---|---|---|
| `propagator.rkt` | Persistent propagator network | `prop-cell`, `propagator`, `prop-netwo` |
| `bilattice.rkt` | Three-valued reasoning (WFS) | `lattice-desc`, `bilattice-var`, `bool` |
| `atms.rkt` | Assumption-based truth maintenance | `assumption`, `supported-value`, `tms-` |
| `tabling.rkt` | SLG-style memoization | `table-entry`, `table-store`, `wf-tabl` |
| `wf-engine.rkt` | Well-founded semantics orchestration | `wf-answer`, `wf-naf-oracle`, bilattice |
| `relations.rkt` | Relation registry and DFS solver | `relation-info`, `solve-goal`, `explai` |
| `solver.rkt` | Solver configuration | `solver-config`, strategy/threshold/t |
| `narrowing.rkt` | Narrowing search (functional-logic bridge) | Value ordering, constraint propagatio |
| `provenance.rkt` | Derivation chain tracking | Explanation trees, derivation depth |

## 2 Mode Prefixes on Logic Variables

### 2.1 Background: What Modes Are

In logic programming, a *mode* declaration specifies the intended data-flow pattern of a predicate's arguments:

- `+` (input): the argument is ground (fully instantiated) on entry
- `-` (output): the argument is free on entry and will be bound by the predicate
- `?` (unknown): the argument may be either ground or free

Mercury pioneered mandatory mode declarations with determinism inference. Ciao Prolog uses a rich assertion language where modes are optional but enable powerful static analysis via CiaoPP. Both systems demonstrate that modes unlock significant optimization opportunities.

### 2.2 Mercury's Mode System

Mercury's execution algorithm exploits modes in several ways:

- **Determinism categories**: From modes, Mercury infers whether a predicate is `det` (exactly one solution), `semidet` (zero or one), `multi` (one or more), or `nondet` (zero or more). Deterministic predicates compile to simple function calls with no backtracking machinery.

- **Clause selection**: For `det=/=semidet` predicates with input arguments, the compiler generates switch statements or hash lookups instead of sequential clause search.
- **Argument reordering**: The compiler may reorder goals within a clause body to ensure input arguments are ground before they are needed, potentially converting `nondet` goals to `semidet` ones.
- **Specialization**: Multiple modes for the same predicate generate separate compiled versions (e.g., `append(+, +, -)` vs `append(-, -, +)`).

### 2.3 Ciao's Assertion Language

Ciao's approach is more flexible: modes are expressed as assertions that serve both as documentation and as targets for static/dynamic verification. CiaoPP infers types, modes, sharing, determinacy, non-failure, and resource bounds.

The assertion language supports properties like:

- `:- pred append(+list, +list, -list).` — mode declaration
- `:- check comp append(X, Y, Z) : (ground(X), ground(Y)) => ground(Z).` — conditional mode

### 2.4 Proposed Prologos Syntax

```
;; Mode-annotated relation
defr append {A : Type}
  | [+xs : List A] [+ys : List A] [-zs : List A]
  ;; Ground inputs, free output -- deterministic append

;; Multi-mode relation
defr member {A : Type}
  | [+x : A] [+xs : List A]         ;; membership test (semidet)
  | [-x : A] [+xs : List A]         ;; enumeration (nondet)

;; Solver can exploit mode information
solve [member ?who '[:alice :bob :carol]]
;; With ?who free and the list ground, the solver enumerates (nondet mode)

solve [member :bob +known-list]
;; With both ground, the solver does a membership test (semidet mode)
```

### 2.5 Determinism Inference from Modes

| Input Modes | Pattern | Determinism | Optimization |
|---|---|---|---|
| All `+` | First-match | `det` | Compile to function call |
| All `+` but last | Generate | `semidet` | Single-solution search, cut on success |
| Mix of `+`, `-` | Search | `multi` | Indexed on `+` args, enumerate `-` args |
| All `-` | Enumerate | `nondet` | Full backtracking search |

### 2.6 Composition with the Propagator Network

Modes interact with Prologos's propagator network in two ways:

1. **Cell initialization**: A `+` argument maps to a propagator cell initialized with a ground value (`net-cell-write` at construction). A `-` argument maps to a cell initialized at `bot`. A `?` argument maps to a cell that may or may not have an initial value.

2. **Propagation direction**: Mode analysis determines which propagators fire first. In a clause like `append(+X, +Y, -Z)`, the propagator that computes `Z` from `X` and `Y` should fire eagerly (forward propagation). In `append(-X, -Y, +Z)`, the reverse propagators decomposing `Z` should fire.

3. **Determinism as lattice metadata**: The determinism category of a goal (`det`, `semidet`, `multi`, `nondet`) can be tracked as metadata on the propagator cell, allowing the scheduler (`run-to-quiescence`) to prioritize deterministic propagations.

### 2.7 Implementation Roadmap

1. Parse mode prefixes (`+`, `-`, `?`) in `defr` parameter lists

2. Extend `param-info` in `relations.rkt` with a `mode` field

3. Add determinism inference in a new `mode-analysis.rkt` module

4. Modify `solve-goal` to select clause ordering and indexing based on modes

5. Generate specialized propagator networks per mode combination

# 3 Domain-Constrained Logic Variables

## 3.1 Background: CLP(X) Family

Classical Prolog treats logic variables as untyped, unifiable terms. The CLP(X) family parameterizes the solver over a constraint domain X:

| Domain | Variables Over | Constraints | Solving Technique |
|---|---|---|---|
| CLP(FD) | Finite integers | `X #> Y`, `X in 1..10` | Arc consistency, propagation |
| CLP(R) | Real numbers | `X > 3.14`, `X + Y =:` 5.0= | Simplex, interval arithmetic |
| CLP(Q) | Rationals | Same as CLP(R) | Exact rational arithmetic |
| CLP(Set) | Finite sets | `X subset Y`, `card(X) = 3` | Set-interval propagation |
| CLP(Bool) | Booleans | `X #\/ Y`, `X #/\ Z` | BDD, SAT |
| CLP(S) | Strings | `concat(X, Y, Z)` | String constraint solvers |

## 3.2 Proposed Prologos Syntax

```
;; Domain-constrained logic variable
defr schedule
  | [?room : FD 1..10] [?time : FD 0..23] [?course : Symbol]

;; Constraints use trait-based dispatch
rel schedule
  | [?r ?t :math101]  :- [?t > 8N] [?t < 17N]
  | [?r ?t :phys201]  :- [?t > 9N] [?t < 18N] [?r /= 5N]

;; Real-valued constraints
defr trajectory
  | [?x : Real] [?y : Real] [?t : Real 0.0..100.0]
  :- [?y = [?x * ?x]] [?t >= 0.0]
```

## 3.3 Arc Consistency and Bounds Propagation as Propagator Cells

The key insight is that CLP constraint propagation *is* propagator network computation:

- An FD variable `?x : FD 1..10` maps to a propagator cell whose value is a domain representation (e.g., a sorted interval list or bitset). The lattice is the *reverse inclusion order* on domains: removing values from the domain is the monotone operation (domain shrinks over time).

- Arc consistency for a binary constraint `c(X, Y)` is a propagator watching `X` and `Y`: when `X`'s domain shrinks, it filters `Y`'s domain (and vice versa). This is exactly a two-input, two-output propagator in the network.

- Bounds propagation is a coarser version: the cell stores only `[lo, hi]` and the propagator updates bounds. This maps to a simpler lattice (pairs of integers with component-wise narrowing).

### 3.4 Lattice Instances for Constraint Domains

These connect directly to the lattice catalog (`docs/research/2026-03-14_LATTICE_CATALOG.org`):

| Domain | Cell Lattice | Merge (`join`) | Bot | Top |
|---|---|---|---|---|
| FD | Reverse powerset | Intersection | Full domain | Empty set (=!) |
| Bounds | Interval `\[lo, hi\]` | Narrow `[max lo, min hi]` | `[-inf, +inf]` | Empty interval |
| Real | Interval arithmetic | Narrow | `(-inf, +inf)` | Empty |
| Set | Set intervals `[lb, ub]` | Narrow both bounds | `[empty, U]` | `lb > ub` |
| Boolean | Trilean `{T, F, unknown}` | Refine | `unknown` | Contradiction |

### 3.5 Composition with ATMS

Domain constraints compose with the ATMS (`atms.rkt`) for speculative reasoning:

```
;; Under different assumptions, a variable may have different domains
solve [schedule ?room ?time :math101]
  assuming { room-available 3 }
  ;; ATMS tracks: {room-available 3} => room domain shrinks to {3}
  ;; Other worldviews may have different domain states
```

Each `supported-value` in a `tms-cell` can carry a domain state. The ATMS manages which domain states are consistent with which assumption sets, enabling the solver to explore multiple constraint scenarios simultaneously.

### 3.6 Implementation Roadmap

1. Define `domain-cell` constructors in `propagator.rkt` (FD, bounds, interval)

2. Implement constraint propagators (arc-consistency, bounds-propagation)
3. Extend `?x:Type` syntax in the relational parser to accept domain annotations
4. Add domain-aware unification in the solver (`unify` + domain narrowing)
5. Connect to the existing `narrowing.rkt` infrastructure

# 4 Co-Inductive Logic Programming and Greatest Fixpoints

## 4.1 Background: Coinduction as Dual of Induction

Standard logic programming computes *least fixpoints* (lfp): start from nothing, build up derivable facts by applying clauses bottom-up until no new facts emerge. Co-inductive logic programming computes *greatest fixpoints* (gfp): start from everything, eliminate non-derivable facts top-down.

The duality:

- **Induction** (lfp): finite derivation trees, well-founded recursion, Herbrand models
- **Coinduction** (gfp): potentially infinite proof trees, corecursion, regular trees

## 4.2 CoLP: Coinductive Hypothesis Rule

Gupta, Bansal, and Simon (2007) introduced CoLP with the *coinductive hypothesis rule*: when proving a goal `G`, if `G` already appears as an ancestor in the current proof search, succeed (coinductive hypothesis) rather than failing (inductive loop detection).

This enables:

```
;; Coinductive definition of infinite streams
:coinductive
defr stream-of
  | [?s : Stream A] [?a : A]

rel stream-of
  | [?s ?a] :- [head ?s ?a] [tail ?s ?rest] [stream-of ?rest ?a]
```

```
  ;; Under coinduction, stream-of [ones 1N] succeeds for ones = 1 :: ones

;; Bisimulation as coinductive equality
:coinductive
defr bisimilar
  | [?s1 : Stream A] [?s2 : Stream A]

rel bisimilar
  | [?s1 ?s2] :- [head ?s1 ?h] [head ?s2 ?h]
                 [tail ?s1 ?t1] [tail ?s2 ?t2]
                 [bisimilar ?t1 ?t2]
```

### 4.3 Flexible Coinduction: Coclauses

Ancona, Barbieri, and Zucca (2020) introduced *coclauses* that allow tuning the coinductive behavior per-predicate. A coclause is matched when a coinductive hypothesis fires, allowing the user to specify *what* should happen when a loop is detected rather than always succeeding.

```
;; Flexible coinduction with coclause
:coinductive
defr subtype
  | [?a : Type] [?b : Type]

;; Regular clauses (inductive base cases)
rel subtype
  | [:Nat :Int]
  | [[List ?a] [List ?b]] :- [subtype ?a ?b]

;; Coclause: when coinductive hypothesis fires, check structural compatibility
coclause subtype
  | [?a ?b] :- [structural-compat ?a ?b]
```

### 4.4 Composition with Bilattice (Well-Founded Engine)

The existing bilattice infrastructure in `bilattice.rkt` already provides the dual ascending/descending cell pairs needed for coinductive reasoning:

- **Ascending cell** (lfp direction): accumulates evidence FOR a predicate being true. This is standard inductive LP.

- **Descending cell** (gfp direction): starts at `top` and eliminates evidence AGAINST a predicate being true. This is the coinductive direction.

The bilattice reading at quiescence gives exactly the three-valued approximation needed:

- `lower = upper = true`: predicate is both inductively and coinductively true
- `lower = false, upper = true`: unknown (gap) — the predicate might be true coinductively but has no finite inductive proof
- `lower = upper = false`: definitively false even coinductively

The `wf-engine.rkt` iterative fixpoint loop already alternates between bottom-up and top-down phases. Extending it with coinductive hypothesis detection requires:

1. Maintaining a *coinductive stack* during DFS proof search
2. When a goal matches an ancestor, consult the bilattice upper cell
3. If upper cell is `true` (not yet refuted), succeed coinductively
4. The refutation phase may later push the upper cell to `false`

### 4.5 Coinductive Session Types

Prologos has session types. Session types are naturally coinductive (a server loop is an infinite protocol). Coinductive LP enables:

```
;; Session type as coinductive predicate
:coinductive
defr conforms
  | [?process : Process] [?session : Session]

rel conforms
  | [?p ?s] :- [step ?p ?action ?p'] [transition ?s ?action ?s']
               [conforms ?p' ?s']
;; Coinductive: the process conforms if it can always make a matching step
```

### 4.6 Implementation Roadmap

1. Add `:coinductive` annotation to `defr` (in `relation-info`, new field)
2. Track coinductive ancestor stack in the DFS solver
3. Connect to bilattice upper cell for coinductive hypothesis resolution
4. Implement coclause syntax and matching
5. Test with session type conformance checking

## 5 Rule Learning and Inductive Logic Programming

### 5.1 Background: ILP Systems

Inductive Logic Programming (ILP) learns logic programs (rules) from examples and background knowledge. The major modern systems:

| System | Method | Language | Key Feature |
|---|---|---|---|
| Metagol | Meta-interpretive | Prolog | Metarule-guided search, predicate invention |
| ILASP | ASP-based | ASP | Optimal w.r.t. cost functions |
| Popper | Hypothesis testing | Prolog + ASP | Generate-test-constrain loop |
| Louise | Polynomial MIL | Prolog | Scalable meta-interpretive learning |
| FastLAS | Fast ILASP variant | ASP | Noise-tolerant |

### 5.2 Meta-Interpretive Learning (MIL)

MIL learns by specializing *metarules*—higher-order clause templates:

```
%% Metarule: chain
metarule(chain, [P,Q,R], [P,A,B] :- [[Q,A,C],[R,C,B]]).
%% "P(A,B) :- Q(A,C), R(C,B)" - learn transitive compositions
```

The hypothesis space is controlled by the set of available metarules. Typed MIL (MetagolT) uses type checking to prune the search space by a cubic factor.

### 5.3 Proposed Prologos Surface Syntax for ILP

```
;; Define a learning task
learn grandparent
  from-examples
    [grandparent :alice :carol] ;; positive
    [grandparent :alice :dave]  ;; positive
  negative-examples
    [grandparent :alice :alice] ;; negative
  background
    parent member
  metarules
    chain identity
  max-clauses 3

;; The system would synthesize:
;; rel grandparent | [?x ?z] :- [parent ?x ?y] [parent ?y ?z]
```

### 5.4 Differentiable ILP

Recent work (NeuralLP, DRUM, and GLIDR) embeds ILP in differentiable frameworks. Key ideas:

- Clause selection is parameterized by continuous weights (softmax over metarules)
- Forward chaining is differentiable (matrix operations over relation adjacency)
- Gradient descent optimizes rule weights against example loss

This connects to Prologos through the propagator network:

- Each candidate clause becomes a propagator with a continuous weight
- The weight lives in a propagator cell over a `\[0,1\]` lattice
- Gradient information flows backward through the network (counter-propagation)
- The ATMS tracks which clause combinations are consistent

```
;; Differentiable ILP as weighted clauses
learn-differentiable path
  from-data edge-triples
  metarules [chain identity inverse]
  optimizer :adam
  epochs 100
  ;; Each learned clause gets a weight:
  ;; rel path | [?x ?y] :- [edge ?x ?y]        ;; weight 0.95
  ;; rel path | [?x ?z] :- [edge ?x ?y] [path ?y ?z]  ;; weight 0.88
```

### 5.5 Composition with Propagator Networks

ILP as propagator computation:

1. **Hypothesis space as lattice**: The set of candidate programs, ordered by generality (more general = higher). Learning descends from the most general hypothesis.

2. **Example propagators**: Each positive/negative example is a propagator that constrains the hypothesis space (positive: must cover; negative: must not cover).

3. **Metarule propagators**: Each metarule instantiation is a propagator that adds a candidate clause to the hypothesis.

4. **Pruning propagators**: Constraint propagation eliminates inconsistent hypothesis subspaces.

The ATMS is particularly valuable here: each candidate clause is an assumption, and nogoods track clause combinations that violate negative examples.

### 5.6 Implementation Roadmap

1. Define `learn` top-level form and metarule syntax

2. Implement metarule-guided search as propagator network construction

3. Connect to ATMS for hypothesis tracking (clauses as assumptions)

4. Add differentiable ILP as a solver strategy (`:optimizer` config)

5. Bridge to the type system (typed MIL uses Prologos's type inference)

# 6 Probabilistic Logic Programming

## 6.1 Background: ProbLog and the Distribution Semantics

ProbLog (KU Leuven) extends Prolog with probabilistic facts:

```
0.3 :: burglary.
0.1 :: earthquake.
alarm :- burglary.
alarm :- earthquake.
```

The *distribution semantics*: each probabilistic fact independently true/false with its stated probability. A ground query's probability is the sum over all possible worlds where it is derivable, weighted by world probability.

Inference uses *knowledge compilation*: the derivation structure is compiled to a Boolean formula (SDD, BDD, or d-DNNF), then weighted model counting computes the probability.

## 6.2 Algebraic ProbLog and Semirings

Algebraic ProbLog (aProbLog) generalizes from probabilities to arbitrary commutative semirings. A semiring `(S, +, x, 0, 1)` replaces:

| Semiring | `+` (combine) | `x` (chain) | Meaning |
|---|---|---|---|
| Probability | `+` | `x` | Standard probabilistic LP |
| Viterbi | `max` | `x` | Most probable proof |
| Tropical | `min` | `+` | Shortest path |
| Boolean | `or` | `and` | Standard LP |
| Fuzzy | `max` | `min` | Fuzzy LP |
| Gradient | `+` | `x` + chain rule | Differentiable inference |
| Access control | `meet` | `join` | Security lattice LP |

This is directly relevant to Prologos: *every semiring-based LP is a propagator network with a specific lattice.*

## 6.3 DeepProbLog: Neural Predicates

DeepProbLog (Manhaeve et al., 2018, with continued development through 2025) extends ProbLog with *neural predicates*: predicates whose truth probability is parameterized by a neural network.

```
nn(digit_net, [X], Y, [0..9]) :: digit(X, Y).
addition(X, Y, Z) :- digit(X, A), digit(Y, B), Z is A + B.
```

The neural network's output is a probability distribution over the predicate's possible values. Learning jointly optimizes the neural network weights and the logical program structure.

### 6.4 DC-ProbLog: Continuous Distributions (2024)

DC-ProbLog (Zuidberghe et al., 2024) extends ProbLog to hybrid discrete-continuous domains using *distributional clauses*. The key contribution is a *measure semantics* that generalizes the distribution semantics to continuous random variables, along with an inference engine (IALW—infinitesimal algebraic likelihood weighting) based on knowledge compilation.

### 6.5 Proposed Prologos Syntax

```
;; Probabilistic facts (discrete)
0.3 :: burglary
0.1 :: earthquake

;; Weighted logic variable
defr diagnosis
  | [?symptom : Symbol] [?disease : Symbol ^?p]
  ;; ?p is the marginal probability of ?disease given ?symptom

;; Distributional clause (continuous)
defr sensor-reading
  | [?sensor : Symbol] [?value : Real ~ Normal ?mu ?sigma]

rel sensor-reading
  | [:thermometer ?v] :- [?v ~ Normal 20.0 2.0]
  | [:barometer ?v]   :- [?v ~ Normal 1013.0 5.0]

;; Query with probability
solve [diagnosis :fever ?d] :probabilistic
;; Returns: {d: :flu, p: 0.7}, {d: :cold, p: 0.25}, ...

;; Conditional probability
solve [alarm] given [earthquake] :probabilistic
```

### 6.6 Semiring Framework as Propagator Lattice

The semiring-based framework maps directly to the propagator network:

- A *probability cell* is a propagator cell over the semiring `(R+, +, x, 0, 1)`. The merge operation is semiring addition (combining evidence from multiple derivation paths).
- A *Viterbi cell* uses `(R+, max, x, 0, 1)` for most-probable-explanation queries. Same propagator structure, different lattice.
- The *gradient semiring* enables backpropagation through the logic program. Each cell stores both a value and its gradient. The merge operation combines values additively and gradients via the chain rule.

New propagator cell types needed:

| Cell Type | Lattice | Merge | Use Case |
|---|---|---|---|
| `prob-cell` | `[0,1]` with `+` | Sum of proofs | Marginal probability |
| `viterbi-cell` | `[0,1]` with `max` | Best proof | MAP inference |
| `log-prob-cell` | `(-inf,0]` with `logadd` | Numerically stable | Large-scale inference |
| `gradient-cell` | `R x R` (value, gradient) | Sum + chain rule | Differentiable LP |
| `interval-cell` | Interval `[lo,hi]` | Narrow | Bounded probability |
| `distribution-cell` | Measure space | Mixture | Continuous distributions |

### 6.7 Knowledge Compilation and the ATMS

ProbLog's inference via knowledge compilation (SDDs, d-DNNF) has a natural counterpart in the ATMS (`atms.rkt`):

- Each probabilistic fact is an *assumption* in the ATMS
- Each derivation path is a *support set* (conjunction of assumptions)
- The probability of a query is the weighted count over consistent support sets
- `atms-solve-all` already enumerates all consistent worldviews
- The probability computation is a post-processing step: sum over worldviews, weighted by product of assumption probabilities

This means probabilistic LP can be implemented as a thin layer over the existing ATMS, without a separate knowledge compilation pipeline.

### 6.8 Implementation Roadmap

1. Add probability annotations to facts (`0.3 :: fact`) in the parser
2. Implement `prob-cell` and `viterbi-cell` lattice types
3. Extend `atms-solve-all` with weighted enumeration
4. Add `:probabilistic` solver strategy in `solver-config`
5. Implement distributional clause syntax and continuous sampling
6. Bridge to the gradient semiring for DeepProbLog-style learning

## 7 Category-Theoretic Foundations

### 7.1 Why Category Theory Matters for Prologos

Prologos already uses categorical structures implicitly:

- Propagator cells form a *category of lattices* with monotone maps as morphisms
- The ATMS manages a *fibered structure* where each worldview is a fiber
- Well-founded semantics uses *approximation fixpoint theory* (Denecker et al.), which is fundamentally about operators on bilattices
- Dependent types form a *locally cartesian closed category*
- Session types form a *category of processes* with session type morphisms

Making these connections explicit enables principled extension and composition.

### 7.2 Toposes as Generalized Logic

A topos is a category that behaves like a generalized universe of sets. Every topos has an *internal logic* that may be intuitionistic, classical, or something more exotic. The subobject classifier `Omega` in a topos generalizes the Boolean truth values `{true, false}` to an arbitrary Heyting algebra.

For Prologos:

- The standard Herbrand model is the topos `Set` (classical logic)

- Three-valued logic (well-founded semantics) corresponds to the topos `Set^{3}` or equivalently presheaves on the three-element chain
- Probabilistic logic corresponds to the *Giry monad* on `Meas` (the category of measurable spaces)
- Fuzzy logic corresponds to sheaves valued in `[0,1]`

A topos-aware logic engine could parameterize over the choice of topos, enabling the same syntax to express classical, intuitionistic, fuzzy, or probabilistic reasoning by swapping the underlying topos.

```
;; Topos-parameterized relation
defr reachable {T : Topos}
  | [?x : Node] [?y : Node] [:via T]

;; Classical: standard LP
solve [reachable :a :b :via Classical]

;; Three-valued: well-founded semantics (already implemented!)
solve [reachable :a :b :via WellFounded]

;; Fuzzy: degree-of-truth in [0,1]
solve [reachable :a :b :via Fuzzy]
```

### 7.3 Sheaves for Contextual Reasoning

Sheaves generalize the idea of "data varying over a space" with *gluing conditions* that ensure local data can be consistently assembled into global data. In programming language terms:

- A presheaf assigns data to each "context" (open set)
- A sheaf adds the condition that compatible local data determines unique global data
- The *sheaf condition* is a coherence requirement

For Prologos, sheaves enable *contextual logic programming*:

```
;; Facts that vary by context (location, time, agent)
defr temperature
```

```
  | [?loc : Location] [?temp : Real] [:context ?ctx]

;; Sheaf condition: overlapping contexts must agree
;; If ctx1 and ctx2 overlap, temperature readings must be consistent

;; Query with context restriction
solve [temperature :kitchen ?t :context :morning]
```

The propagator network implements the sheaf gluing condition: consistency propagators between overlapping contexts ensure the sheaf axiom holds.

### 7.4 Kan Extensions for Program Transformation

Kan extensions formalize the notion of "best approximation" when changing the domain of a functor. In programming:

- *Left Kan extension* corresponds to existential quantification / colimit / "the best summary"
- *Right Kan extension* corresponds to universal quantification / limit / "the best conservative approximation"

For Prologos, Kan extensions relate to:

1. **Abstract interpretation**: The abstraction function `alpha` is a left adjoint (left Kan extension), and the concretization `gamma` is a right adjoint. This is a Galois connection.
2. **Program optimization**: The continuation-passing-style transformation corresponds to the codensity monad, which arises from a right Kan extension. This connects to Prologos's narrowing infrastructure.
3. **Query optimization**: Magic set transformations (converting bottom-up evaluation to goal-directed) can be expressed as Kan extensions along the inclusion functor from relevant facts to all facts.

### 7.5 Galois Connections for Abstract Interpretation

A Galois connection `<alpha, C, D, gamma>` between a concrete domain `C` and an abstract domain `D` provides:

- `alpha: C -> D` (abstraction): maps concrete values to their best abstract approximation
- `gamma: D -> C` (concretization): maps abstract values to the concrete values they represent
- Soundness guarantee: `alpha . gamma` < $\mathrm{id}_D$= and `id_C` < gamma . alpha=

In Prologos's propagator framework:

```
;; Abstract interpretation as cross-domain propagation
;; (already have net-add-cross-domain-propagator in propagator.rkt!)

;; Concrete domain: exact integer sets
;; Abstract domain: sign lattice {neg, zero, pos, top, bot}

;; The Galois connection is a pair of cross-domain propagators:
;; alpha: integer-set-cell -> sign-cell (abstraction propagator)
;; gamma: sign-cell -> integer-set-cell (concretization propagator)
```

The existing `net-add-cross-domain-propagator` in `propagator.rkt` is already the infrastructure needed for Galois-connected abstract interpretation.

### 7.6 Fibered Categories for Dependent Types + Logic

Prologos has both dependent types and logic programming. The categorical framework that unifies these is *fibered category theory* (Jacobs, 1999):

- A *fibration* `p: E -> B` maps a "total" category `E` to a "base" category `B`
- Each object `b` in `B` has a *fiber* `E_b` (the category of types/propositions depending on `b`)
- Reindexing along morphisms in `B` gives substitution
- The Curry-Howard-Lambek correspondence extends: types are objects in fibers, terms are morphisms, dependent types are fibered structures

For Prologos, this means:

- The type `<(x : Nat) -> Vec x A>` lives in a fiber over `Nat`
- A relational query `solve [vec-append ?xs ?ys ?zs]` where `?xs : Vec n A` is a morphism in the fiber over `n`
- Substitution of `n : 3`= reindexes the fiber, specializing the query

## 7.7 Institutions for Multi-Logic Reasoning

Institutions (Goguen and Burstall, 1992) formalize the notion of a "logical system" abstractly, enabling reasoning across multiple logics:

- An institution `I = (Sig, Sen, Mod, |)`= specifies:
  - `Sig`: category of signatures
  - `Sen`: functor from signatures to sentences
  - `Mod`: functor from signatures to models (contravariant)
  - `|=`: satisfaction relation (parameterized by signature)

For Prologos, institutions enable multi-logic queries:

```
;; A query that mixes classical and well-founded reasoning
solve
  [parent :alice ?x]                    ;; classical LP
  :and
  (wf [reachable ?x :goal])            ;; well-founded semantics
  :and
  (prob [risky ?x] > 0.5)              ;; probabilistic LP
```

Each sub-query runs in its own "institution" (logic), and the institution morphisms (signature translations) ensure consistency across the boundary. The propagator network is the *universal medium*: each institution maps to a specific lattice/propagator configuration, and cross-domain propagators implement the institution morphisms.

## 7.8 Implementation Roadmap

1. Make lattice descriptors (`lattice-desc`) a first-class abstraction parameterizable by topos choice
2. Implement sheaf gluing as consistency propagators

3. Express abstract interpretation via cross-domain propagators + Galois connections
4. Document the fibered structure of dependent types + relational sublanguage
5. Design institution-aware multi-logic solver dispatch

# 8 Novel Directions

## 8.1 Linear Logic Programming

Prologos already has QTT (Quantitative Type Theory) tracking resource usage via multiplicities (`m0`, `m1`, `mw`). Extending this to the relational sublanguage enables *linear logic programming* where facts are *consumed* by derivation.

```
;; Linear fact: consumed when used
defr has-item {1}  ;; multiplicity 1 = linear
  | [?agent : Agent] [?item : Item]

;; Using a linear fact consumes it
rel craft
  | [?agent :sword] :- {1} [has-item ?agent :iron]
                       {1} [has-item ?agent :wood]
  ;; After this derivation, agent no longer has iron or wood

;; Unrestricted fact: can be used any number of times
defr knows {w}
  | [?agent : Agent] [?recipe : Recipe]
```

The propagator network needs a new cell type for linear resources: a *counting cell* where the merge operation is addition (resource accumulation) and consumption is subtraction (resource use). The lattice is `Z` (integers) with a "non-negative" invariant.

Recent work (2024) on deadlock-free separation logic and the Linear Session Abstract Machine (SAM) shows how linear logic, session types, and concurrency compose, which is directly relevant to Prologos's session type system.

## 8.2 Temporal Logic Programming

Express LTL (Linear Temporal Logic) and CTL (Computation Tree Logic) properties as relational queries.

```
;; Temporal operators as relational combinators
defr eventually
  | [?prop : Prop] [?trace : Trace]

rel eventually
  | [?p ?t] :- [holds ?p [head ?t]]
  | [?p ?t] :- [eventually ?p [tail ?t]]

defr always
  | [?prop : Prop] [?trace : Trace]

:coinductive  ;; Greatest fixpoint!
rel always
  | [?p ?t] :- [holds ?p [head ?t]] [always ?p [tail ?t]]

;; CTL: exists-path and forall-path over branching traces
defr exists-eventually
  | [?prop : Prop] [?state : State]

rel exists-eventually
  | [?p ?s] :- [holds ?p ?s]
  | [?p ?s] :- [transition ?s ?s'] [exists-eventually ?p ?s']
```

Note that `always` is naturally coinductive (gfp), while `eventually` is inductive (lfp). The bilattice infrastructure handles both directions simultaneously.

Temporal LP composes with the propagator network through *trace cells*: a cell whose lattice is a prefix ordering on execution traces. The `run-to-quiescence` scheduler already handles iterative fixpoints over such lattices.

## 8.3 Epistemic Logic Programming

Knowledge and belief operators for multi-agent reasoning:

```
;; Epistemic operators
defr knows
```

```
  | [?agent : Agent] [?prop : Prop]

defr believes
  | [?agent : Agent] [?prop : Prop]

;; Common knowledge
defr common-knowledge
  | [?group : List Agent] [?prop : Prop]

;; Epistemic rules
rel knows
  | [:alice [safe ?code]] :- [observed :alice ?code]
  | [?a ?p]               :- [told ?a ?p ?source] [trusts ?a ?source]

;; Negative introspection: if alice doesn't know p, she knows she doesn't
rel knows
  | [?a [not-known ?p]] :- (not [knows ?a ?p])
```

Each agent's knowledge state maps to a separate *worldview* in the ATMS. The ATMS assumption sets model possible worlds: agent `A` knows `P` iff `P` holds in all worldviews consistent with `A`'s observations.

### 8.4 Abductive Logic Programming

Abduction explains observations by hypothesizing causes:

```
;; Abducible predicates (can be hypothesized)
:abducible
defr broken | [?component : Component]

:abducible
defr leak | [?pipe : Pipe]

;; Rules connecting causes to effects
rel alarm
  | [] :- [broken :sensor]
  | [] :- [leak :main-pipe] [not [valve-closed :main]]

;; Abductive query: what explains the observation?
abduce [alarm]
;; Returns: {broken :sensor} OR {leak :main-pipe, not valve-closed :main}
```

The ATMS is the natural implementation vehicle: each abducible fact becomes an assumption. `atms-solve-all` enumerates minimal consistent explanations. The `atms-minimal-diagnoses` function already implements GDE-2 style minimal diagnosis computation, which is precisely abductive reasoning.

### 8.5 Answer Set Programming (ASP) Integration

ASP extends LP with stable model semantics, choice rules, and aggregates:

```
;; Choice rule: non-deterministically include/exclude
choice [color ?node ?c] :- [node ?node] [c in '[:red :green :blue]]

;; Integrity constraint: no two adjacent nodes same color
:- [edge ?n1 ?n2] [color ?n1 ?c] [color ?n2 ?c]

;; Aggregate
defr total-cost | [?cost : Nat]
rel total-cost
  | [?total] :- [?total = #sum { ?c : [task ?t] [cost ?t ?c] }]
```

ASP's stable models correspond to the ATMS's consistent worldviews:

- Each choice rule creates an `atms-amb` (mutually exclusive assumptions)
- Integrity constraints become `atms-add-nogood` (forbidden assumption sets)
- `atms-solve-all` enumerates stable models

The connection between ASP and CHR has been established (Abdennadher et al., 2021): first-order ASP programs can be compiled to CHR programs, eliminating the grounding phase.

### 8.6 Constraint Handling Rules (CHR)

CHR is a committed-choice rule language for implementing constraint solvers:

```
;; CHR rules for a simple constraint solver
;; Simplification: replace constraint with simpler one
chr leq-reflexive
  | [leq ?x ?x] <=> :true
```

```
;; Propagation: add new constraint without removing old
chr leq-transitive
  | [leq ?x ?y] [leq ?y ?z] ==> [leq ?x ?z]

;; Simpagation: some kept, some removed
chr leq-antisymmetric
  | [leq ?x ?y] \ [leq ?y ?x] <=> [?x = ?y]
```

CHR maps directly to propagator networks:

- Each constraint is a propagator cell
- Simplification rules are propagators that replace cell values
- Propagation rules add new cells and propagators
- The CHR store is the propagator network state

Recent work on Quantified CHR (QCHR, 2025) extends CHR with dynamic quantifier generation, which could interact with Prologos's dependent types.

## 8.7 Advanced Tabling: Beyond SLG

Prologos has SLG-style tabling (`tabling.rkt`). Extensions to explore:

### 8.7.1 Subsumptive Tabling

Standard variant tabling memoizes exact call patterns. Subsumptive tabling recognizes that a more general call subsumes a more specific one:

```
;; With subsumptive tabling:
;; solve [ancestor :alice ?x] produces answers for all descendants
;; solve [ancestor :alice :carol] can reuse the table entry
;; (because ancestor(:alice, ?x) subsumes ancestor(:alice, :carol))
```

### 8.7.2 Mode-Directed Tabling

Combine modes with tabling to control which answers are kept:

```
;; Keep only the minimum-cost path (mode-directed)
:- table shortest_path(+, +, min)
defr shortest-path
  | [+from : Node] [+to : Node] [-cost : Nat :aggregate min]
```

### 8.7.3 Magic Sets

Transform bottom-up evaluation to be goal-directed (top-down efficiency with bottom-up termination guarantees). This is particularly relevant for Datalog-style reasoning within Prologos.

## 8.8 Implementation Priority Matrix

| Extension | Infrastructure Reuse | New Lattices | New Syntax | Complexity |
|---|---|---|---|---|
| Linear LP | QTT multiplicities | Counting | Moderate | Medium |
| Temporal LP | Bilattice (co/ind) | Trace prefix | Moderate | Medium |
| Epistemic LP | ATMS worldviews | None | Light | Low |
| Abductive LP | ATMS diagnoses | None | Light | Low |
| ASP integration | ATMS (amb, nogood) | None | Moderate | Medium |
| CHR | Propagator network | None | Heavy | High |
| Subsumptive tabling | tabling.rkt | None | None | Medium |
| Mode-directed tab. | tabling.rkt + modes | Aggregate | Light | Medium |
| Magic sets | tabling.rkt | None | None | High |

# 9 The Frontier: Modern LP Redesign (2020–2026)

## 9.1 Survey of Recent Systems

### 9.1.1 Flix (Aarhus/Waterloo, 2016–2025)

Flix extends Datalog with arbitrary lattices and monotone functions, integrated as a full programming language. Key innovations:

- *Lattice semantics*: Datalog relations can map to lattice values, not just sets of tuples. Computation proceeds to the least fixpoint over the lattice.
- *First-class Datalog programs*: Functions can take and return Datalog programs as values (rho abstraction), enabling modular analysis composition.
- *Type classes*: Flix uses type classes for lattice operations, paralleling Prologos's trait system.
- *Stratified negation + lattice fixpoints*: Combines standard Datalog negation with lattice-valued computation.

Relevance to Prologos: Flix demonstrates that lattice-parameterized fixpoint computation (exactly what Prologos's propagator network does) is a viable foundation for a full language. Prologos goes further by adding backtracking search, ATMS, and well-founded semantics.

#### 9.1.2 Formulog (Harvard, 2020–2024)

Formulog = Datalog + SMT + ML-like functions. Key innovations:

- *SMT integration*: Datalog rules can contain SMT constraints; the solver dispatches to Z3/CVC5 for satisfiability checking.
- *Eager evaluation*: A 2024 paper (Bembenek et al., OOPSLA 2024) shows that DFS-style eager evaluation achieves 5–8x speedups over Soufflé-style semi-naive evaluation for SMT-heavy workloads.
- *Algebraic data types*: First-class ADTs in Datalog, enabling analysis of languages with recursive data.

Relevance to Prologos: Formulog's SMT integration could be modeled as an external constraint domain (like CLP(SMT)). The eager evaluation strategy is interesting for Prologos's solver, which already uses DFS.

#### 9.1.3 Ascent (2022–2024)

Ascent embeds Datalog in Rust via macros, with:

- Semi-naive evaluation compiled to Rust
- Lattice support (similar to Flix)
- Incremental computation

#### 9.1.4 Crepe (2021–2024)

Crepe provides Datalog as a Rust procedural macro:

- Semi-naive evaluation with stratified negation
- Seamless Rust interop
- Compile-time Datalog analysis

### 9.1.5 Flan (Purdue, POPL 2024)

Flan embeds Datalog in Scala via multi-stage programming:

- Combines Flix's flexibility with Soufflé's performance
- Arbitrary aggregates, user-defined functions, lattices
- Seamless host language integration

### 9.1.6 Soufflé (2016–2025)

High-performance Datalog compiler:

- Compiles to parallel C++
- Used in production for program analysis (Java points-to, security)
- Subsumptive tabling, aggregates, ADTs

## 9.2 What These Systems Teach Us

| Capability | Classic Prolog | Flix | Formulog | Soufflé | Prologos (current) | Prologos ( |
|---|---|---|---|---|---|---|
| Backtracking search | Yes | No | No | No | Yes | Yes |
| Lattice values | No | Yes | No | Partial | Yes (propagator) | Yes |
| SMT integration | No | No | Yes | No | No | Via CLP( |
| Dependent types | No | No | No | No | Yes | Yes |
| Session types | No | No | No | No | Yes | Yes |
| Linear types (QTT) | No | No | No | No | Yes | Yes |
| Well-founded semantics | XSB | No | No | No | Yes (bilattice) | Yes |
| ATMS/TMS | No | No | No | No | Yes | Yes |
| Probabilistic | ProbLog ext. | No | No | No | No | Yes (semi |
| Coinductive | CoLP ext. | No | No | No | No | Yes (bilat |
| Modes | Mercury ext. | No | No | No | No | Yes |
| ILP | Metagol ext. | No | No | No | No | Yes (ATM |
| Incremental | No | No | No | Partial | No | Via propa |

The table reveals Prologos's unique position: it is the only system that combines backtracking search with lattice-based propagators, dependent types, and ATMS-based assumption management. The proposed extensions would make it the most expressive logic programming system in existence.

### 9.3 The Propagator Network as Universal Substrate

The central thesis of this document: *every extension of logic programming maps to a specific configuration of the propagator network.*

| LP Extension | Propagator Configuration |
|---|---|
| Standard LP | Boolean cells, unification propagators, DFS scheduler |
| CLP(FD) | Domain cells, arc-consistency propagators |
| Well-founded LP | Bilattice cells, alternating fixpoint scheduler |
| Probabilistic LP | Probability cells (semiring), knowledge compilation via ATMS |
| Coinductive LP | Descending cells (gfp), coinductive hypothesis propagators |
| ASP | ATMS assumptions as choices, nogoods as integrity constraints |
| Abductive LP | ATMS assumptions as abducibles, minimal diagnosis |
| Fuzzy LP | `[0,1]` cells, fuzzy connective propagators |
| Temporal LP | Trace-prefix cells, unfolding propagators |
| Linear LP | Counting cells, resource-consumption propagators |
| Abstract interp. | Galois-connected cross-domain propagators |
| Datalog/Flix-style | Lattice cells, semi-naive evaluation as propagation strategy |

This is not a coincidence. The Radul-Sussman propagator model is a *generalization* of constraint propagation, which itself generalizes all of the above. Prologos's architecture makes this explicit and exploitable.

## 10 Composition Map

How do these extensions compose with each other and with existing Prologos features?

### 10.1 Pairwise Composition Matrix

| | Modes | Domains | Coinduction | ILP | Probabilistic | Category | Linear |
|---|---|---|---|---|---|---|---|
| **Modes** | — | Strong | Moderate | Weak | Moderate | Strong | Strong |
| **Domains** | | — | Weak | Weak | Strong | Strong | Moderate |
| **Coinduction** | | | — | Weak | Moderate | Strong | Moderate |
| **ILP** | | | | — | Strong | Weak | Weak |
| **Probabilistic** | | | | | — | Strong | Moderate |
| **Category** | | | | | | — | Strong |
| **Linear** | | | | | | | — |

Legend: Strong = natural synergy, Moderate = composable with some work, Weak = mostly independent.

## 10.2 Key Composition Synergies

### 10.2.1 Modes + Domains (Strong)

Mode analysis determines propagation direction; domain constraints determine what propagates. Together: mode `(+)` on a domain variable means the domain is fully known at entry; mode `(-)` means the domain must be computed. This enables the constraint solver to select arc-consistency (for `?` mode) vs. direct evaluation (for `+` mode).

### 10.2.2 Probabilistic + Domains (Strong)

CLP(FD) with probabilities = probabilistic constraint satisfaction. The probability cell tracks the probability of each domain value, enabling marginal inference over constrained variables. DC-ProbLog already combines distributional clauses with constraint domains.

### 10.2.3 Probabilistic + ILP (Strong)

Learning probabilistic logic programs. ProbLog + ILP = structure learning for probabilistic models. The ATMS tracks assumption probabilities, and the gradient semiring enables differentiable structure learning.

### 10.2.4 Coinduction + Category Theory (Strong)

Coinductive types are terminal coalgebras; coinductive LP computes greatest fixpoints. The categorical framework provides the theory (final coalgebra semantics), and coinductive LP provides the computation. Together: coinductive reasoning about infinite structures with categorical guarantees.

### 10.2.5 Modes + Linear Types (Strong)

Mode analysis tells you which arguments are consumed (`+` in linear context) and which are produced (`-`). This determines the resource flow. QTT multiplicities (`m0`, `m1`, `mw`) already track this at the type level; modes extend it to the relational level.

### 10.2.6 Category Theory + Probabilistic (Strong)

The Giry monad provides the categorical foundation for probabilistic programming. Semiring-based LP is a functor from the category of semirings to the category of LP semantics. The propagator network is the *universal construction* that internalizes this functor.

## 10.3 Composition with Existing Prologos Features

### 10.3.1 With Dependent Types

Every extension must respect the type system. Modes become *type-level mode annotations*. Domain constraints become *refinement types*. Probabilities become *graded modalities*. The fibered category framework ensures coherence.

### 10.3.2 With Session Types

Session types are coinductive protocols. Probabilistic session types (Das & Pfenning, 2023) track expected resource usage. Linear session types ensure protocol adherence. Modes on session types determine whether a channel is for sending (`+`) or receiving (`-`).

### 10.3.3 With the Propagator Network

All extensions are *propagator network configurations*. The network is the universal substrate. New extensions add:

- New cell types (domain cells, probability cells, trace cells, counting cells)
- New propagator types (arc-consistency, gradient, coinductive hypothesis)
- New scheduler strategies (semi-naive, alternating fixpoint, belief propagation)
- New lattice descriptors (semirings, interval lattices, bilattices)

### 10.3.4 With the ATMS

The ATMS is the universal *assumption management* layer:

- Probabilistic LP: assumptions with probabilities

- Abductive LP: assumptions as abducibles
- ASP: assumptions as choices, nogoods as constraints
- ILP: assumptions as candidate clauses
- Epistemic LP: assumptions as agent observations

# 11 Research References

## 11.1 Mode Analysis and Optimization


- Somogyi, Henderson, Conway. "The Execution Algorithm of Mercury." JLP, 1996.
- Overton, Somogyi, Stuckey. "Constraint-Based Mode Analysis of Mercury." PPDP, 2002.
- Hermenegildo et al. "An Overview of the Ciao Multiparadigm Language and Program Development Environment." ICLP, 2012.
- Stade et al. "A Practical Approach for Testing Static Analysis Truths." 2025.


## 11.2 Coinductive Logic Programming


- Simon, Mallya, Bansal, Gupta. "Coinductive Logic Programming." ICLP, 2006.
- Gupta, Bansal, Simon. "Coinductive Logic Programming and Its Applications." ICLP, 2007.
- Ancona, Barbieri, Zucca. "Flexible Coinductive Logic Programming." TPLP, 2020.
- Ancona, Barbieri, Zucca. "Checking Equivalence of Corecursive Streams." 2024.


## 11.3 Probabilistic Logic Programming


- De Raedt, Kimmig, Toivonen. "ProbLog: A Probabilistic Prolog." UAI, 2007.

- Manhaeve, Dumancic, Kimmig, Demeester, De Raedt. "DeepProbLog: Neural Probabilistic Logic Programming." NeurIPS, 2018.
- Derkinderen, Manhaeve et al. "The DeepLog Neurosymbolic Machine." 2025.
- Zuidberghe et al. "Declarative Probabilistic Logic Programming in Discrete-Continuous Domains." AIJ, 2024.
- "Semirings for Probabilistic and Neuro-Symbolic Logic Programming." 2024.


### 11.4 Inductive Logic Programming


- Muggleton, Lin. "Meta-interpretive Learning of Higher-Order Dyadic Datalog." MLJ, 2014.
- Cropper, Morel. "Learning Programs by Learning from Failures" (Popper). MLJ, 2020.
- Cropper, Dumancic, Evans, Muggleton. "Inductive Logic Programming at 30." MLJ, 2021.
- Morel, Cropper. "Typed Meta-interpretive Learning of Logic Programs." ILP, 2019.
- "Meta-Interpretive Learning with Reuse." Mathematics, 2024.
- "GLIDR: Graph-Like Inductive Logic Programming with Differentiable Reasoning." 2025.
- Rocha et al. "Program Synthesis Using ILP for the ARC." 2025.
- "Bridging Logic Programming and Deep Learning for Explainability through ILASP." 2025.


### 11.5 Category Theory and Logic


- Jacobs. "Categorical Logic and Type Theory." Elsevier, 1999.
- Goguen, Burstall. "Institutions: Abstract Model Theory for Specification and Programming." JACM, 1992.
- Hinze. "Kan Extensions for Program Optimisation." MPC, 2012.

- "Category-Theoretical and Topos-Theoretical Frameworks in Machine Learning: A Survey." Axioms, 2025.
- Shiebler. "Kan Extensions in Data Science and Machine Learning." 2022.


### 11.6 Well-Founded Semantics and Bilattices


- Fitting. "Bilattices and the Semantics of Logic Programming." FACS, 1991.
- Van Gelder, Ross, Schlipf. "The Well-Founded Semantics for General Logic Programs." JACM, 1991.
- Denecker, Marek, Truszczynski. "Approximation Fixpoint Theory." 2004.
- Charalambidis et al. "The Stable Model Semantics for Higher-Order Logic Programming." 2024.


### 11.7 Modern Datalog Systems


- Madsen, Yee, Lhotak. "From Datalog to Flix." PLDI, 2016.
- Bembenek, Greenberg, Chong. "Formulog: Datalog for SMT-Based Static Analysis." OOPSLA, 2020.
- Bembenek, Greenberg, Chong. "Making Formulog Fast." OOPSLA, 2024.
- "Flix: A Design for Language-Integrated Datalog." PACMPL, 2025.
- Abeysinghe et al. "Flan: An Expressive and Efficient Datalog Compiler." POPL, 2024.


### 11.8 Linear Logic and Session Types


- Caires, Pfenning. "Session Types as Intuitionistic Linear Propositions." CONCUR, 2010.
- "Comparing Session Type Systems Derived from Linear Logic." 2024.
- "The Linear Session Abstract Machine." 2024.

- Das, Pfenning. "Probabilistic Resource-Aware Session Types." POPL, 2023.
- "Deadlock-Free Separation Logic: Linearity Yields Progress." POPL, 2024.
- Derakhshan, Pfenning. "Rast: Resource-Aware Session Types with Arithmetic Refinements." FSCD, 2020.


### 11.9 ASP and CHR


- Abdennadher et al. "First-Order ASP Programs as CHR Programs." SAC, 2021.
- "Quantified Constraint Handling Rules." TOCL, 2025.


### 11.10 Abductive Logic Programming


- "An Efficient Propositional System for Abductive Logic Programming." AI Review, 2024.


### 11.11 Tabling


- Swift, Warren. "XSB: Extending Prolog with Tabled Logic Programming." TPLP, 2012.
- Warren. "Programming in Tabled Prolog." Draft, 2023.


## 12 Appendix: Lattice Requirements per Extension

| Extension | Cell Lattice | Direction | Merge | Needs New `lattice-desc`? |
|---|---|---|---|---|
| Standard LP | Boolean | Ascending | Disjunction | No (`bool-lattice`) |
| WFS | Boolean bilattice | Both | Join/Meet | No (`bilattice-var`) |
| CLP(FD) | Reverse powerset | Descending | Intersection | Yes |
| CLP(Bounds) | Interval | Descending | Narrow | Yes |
| Probabilistic | `[0,1]` semiring | Ascending | Addition | Yes |
| Viterbi | `[0,1]` max-semiring | Ascending | Max | Yes |
| Fuzzy | `[0,1]` Heyting | Ascending | Max | Yes |
| Coinductive | Boolean | Descending | Conjunction | No (reuse `bool-lattice`) |
| Linear resource | `Z` (integers) | Ascending | Addition | Yes |
| Trace prefix | Prefix order | Ascending | Common prefix | Yes |
| Sign abstract | Sign lattice | Ascending | Join | Yes |
| Set interval | Pair of sets | Both | Narrow bounds | Yes |

# 13 Appendix: Implementation Priority

Based on infrastructure reuse, expressive power gained, and implementation complexity:

## 13.1 Tier 1: Low-Hanging Fruit (High Reuse, Moderate Effort)

1. **Abductive LP** — Almost free: ATMS assumptions as abducibles, `atms-minimal-diagnoses` as explanation engine. Needs only surface syntax.

2. **Epistemic LP** — ATMS worldviews as agent knowledge states. Surface syntax + thin dispatch layer over existing `atms-solve-all`.

3. **ASP Integration** — ATMS `amb` for choice rules, `nogood` for integrity constraints. The hardest part is the surface syntax for aggregates.

## 13.2 Tier 2: Medium Effort, High Impact

1. **Mode Analysis** — Significant optimization potential. Requires new analysis pass but no new propagator infrastructure.

2. **Coinductive LP** — Reuses bilattice descending cells. Requires coinductive hypothesis detection in the DFS solver and coclause syntax.

3. **Domain-Constrained Variables (CLP(FD))** — New lattice types but standard propagator infrastructure. High practical value for constraint satisfaction.

### 13.3 Tier 3: High Effort, Transformative Impact

1. **Probabilistic LP** — New semiring lattices, probability computation, and potentially knowledge compilation. Very high expressive payoff.

2. **CHR** — Full committed-choice rule engine. High implementation cost but provides a meta-level constraint programming capability.

3. **ILP/Rule Learning** — Requires metarule search, hypothesis management, and potentially differentiable infrastructure.

### 13.4 Tier 4: Research-Grade

1. **Category-theoretic topos parameterization** — Foundational but speculative. Provides the theoretical framework for all other extensions.

2. **Institution-based multi-logic dispatch** — Enables mixing logics in a single query. Requires significant design work.

3. **Temporal LP** — Useful for verification but requires trace infrastructure and careful interaction with the scheduler.